Introductory Programming Techniques

North Metropolitan TAFE

ICTPRG302: Session 9

Courses:

- Certificate III in Information Technology
- Certificate IV in IT (Programming)
- Certificate IV in IT (Networking)
- Certificate IV in IT (Cybersecurity)
- Certificate IV in IT (Web)

What will I learn?

- How to open, read, write and close files in Python
- Key functions like open(), read(), write() and close()
- Best practices for managing files
- Techniques for handling data storage and retrieval.

To complete this week

- Weekly session exercises
- Portfolio questions
- Revision…practise, practise, practise!

Revision: Dictionaries

What can you remember? **True** or **False?**

In a Python dictionary, keys must be unique, but values can be duplicated.

Revision: Dictionaries

What can you remember? **True** or **False?**

Dictionaries in Python maintain the order of items as they are added.

Revision: Dictionaries

What can you remember? **True** or **False?**

The get() method in a dictionary returns the value for a specified key, and raises a KeyError if the key is not found.

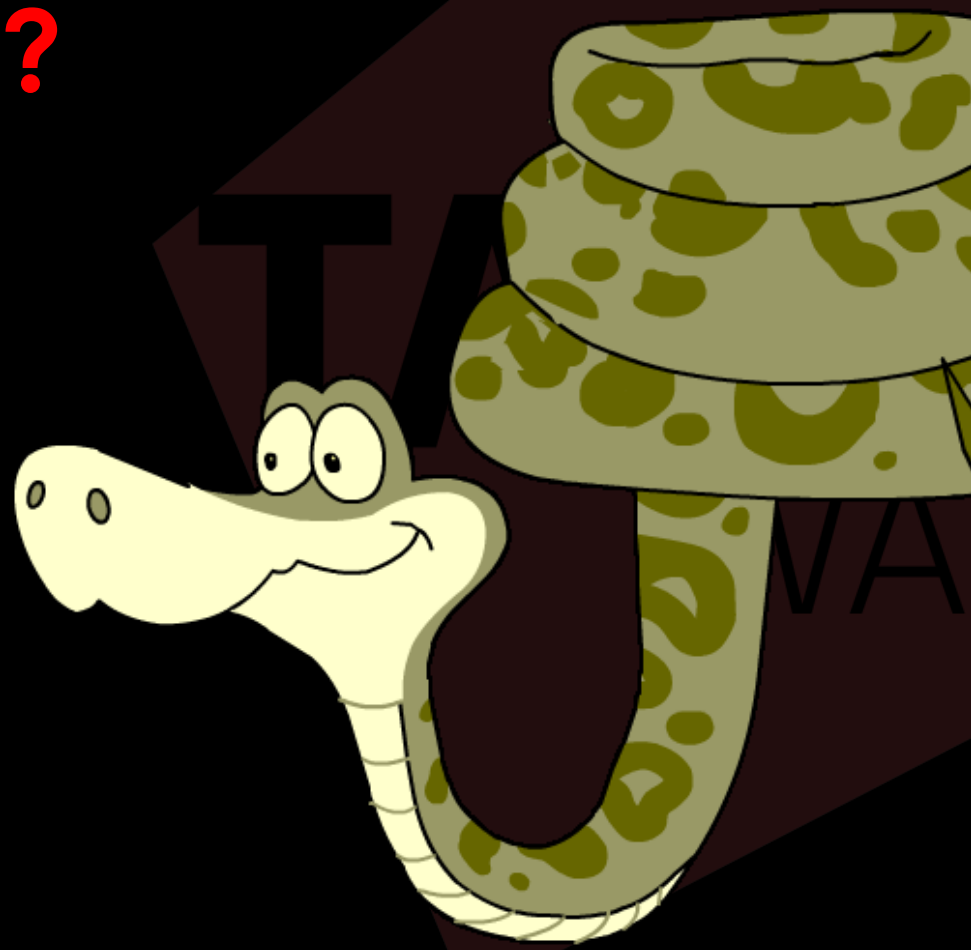
Reminder: Additional resources

Freely available online courses, that can be useful to enhance your knowledge.

- Python for Everyone (Py4e)
- LearnPython.org
- W3schools

What is a File?

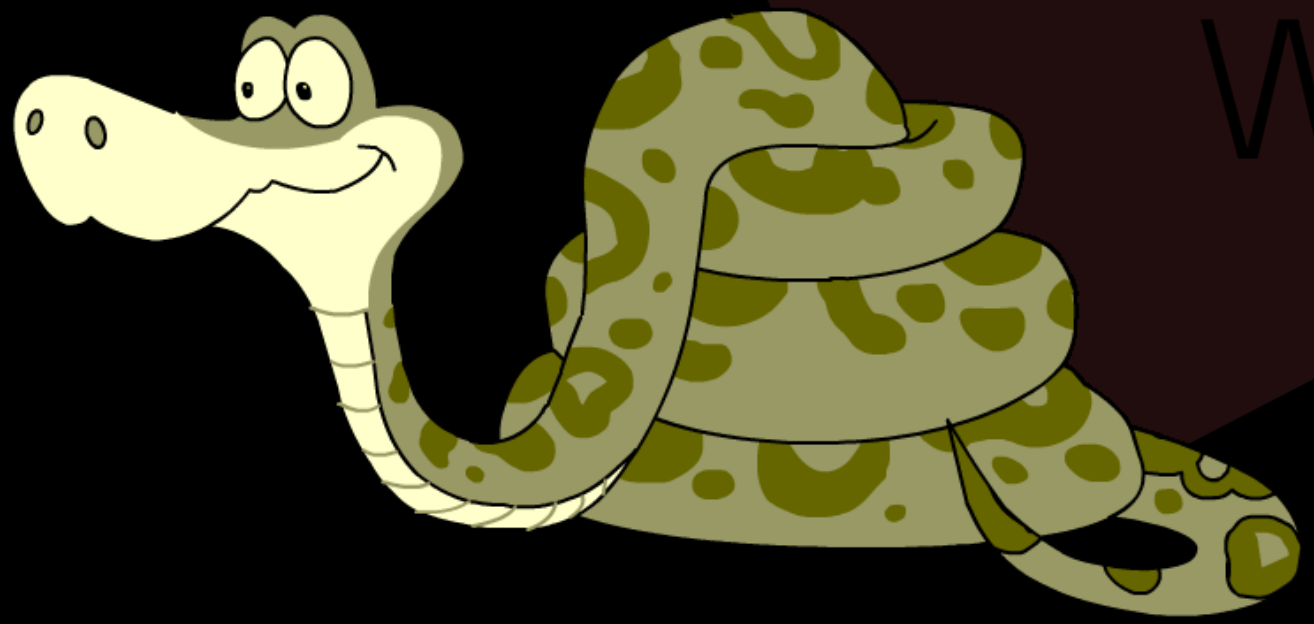
A **file** is a term used to describe data that is held in storage on a computer.



File Types

Files come in many forms, for example:

- Text files
- Image files
- Video files
- Sound files.



File Usage

- Files can be used by multiple programs for different purposes.
- Allows data to be used multiple times.

The Importance of File Handling

- Store and retrieve data
- Process large amounts of data
- Read configuration files
- Save program output.

File handling

A text file might be used to keep track of the highest score for a game.

Allows the highest score to be stored and this can be accessed and modified each time the game is played.

File Handling

Another example is when a program can access large data sets.

A weather station can use sensors to take weather readings that are stored in a data file.

A program can then analyse the data in the file to make predictions about the weather.

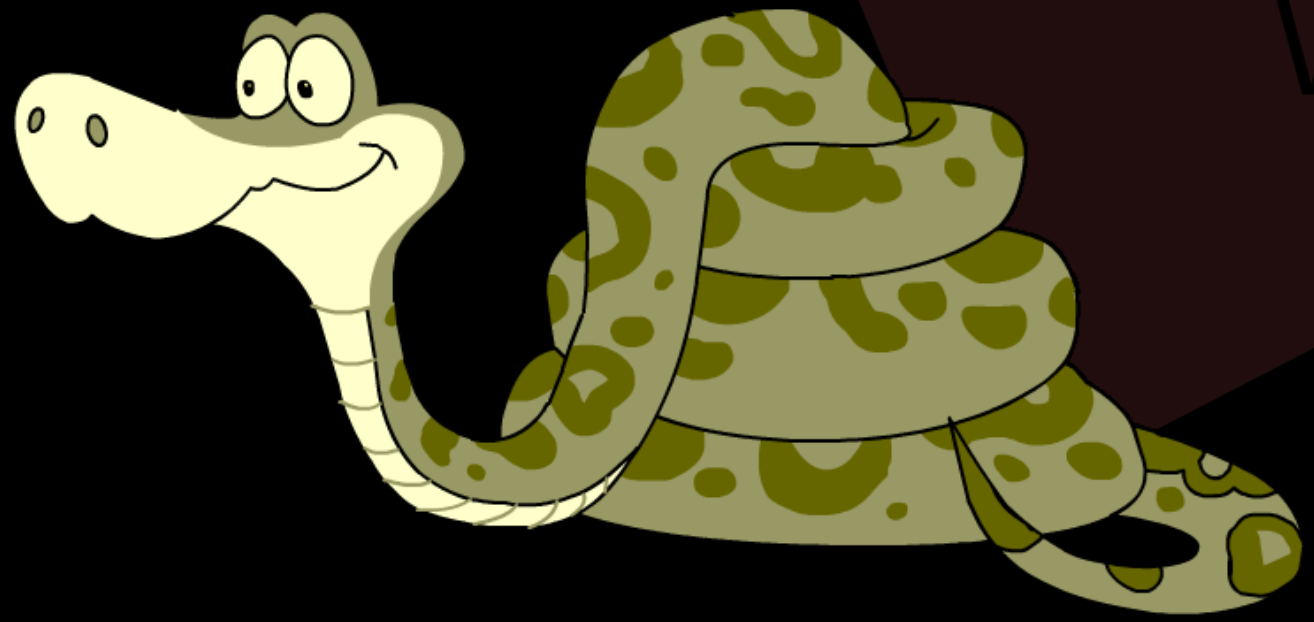
File Handling

Can you think of any other examples of when file-handling might be used?

File Handling in Python

The key **file-handling techniques** that you need to be able to use are:

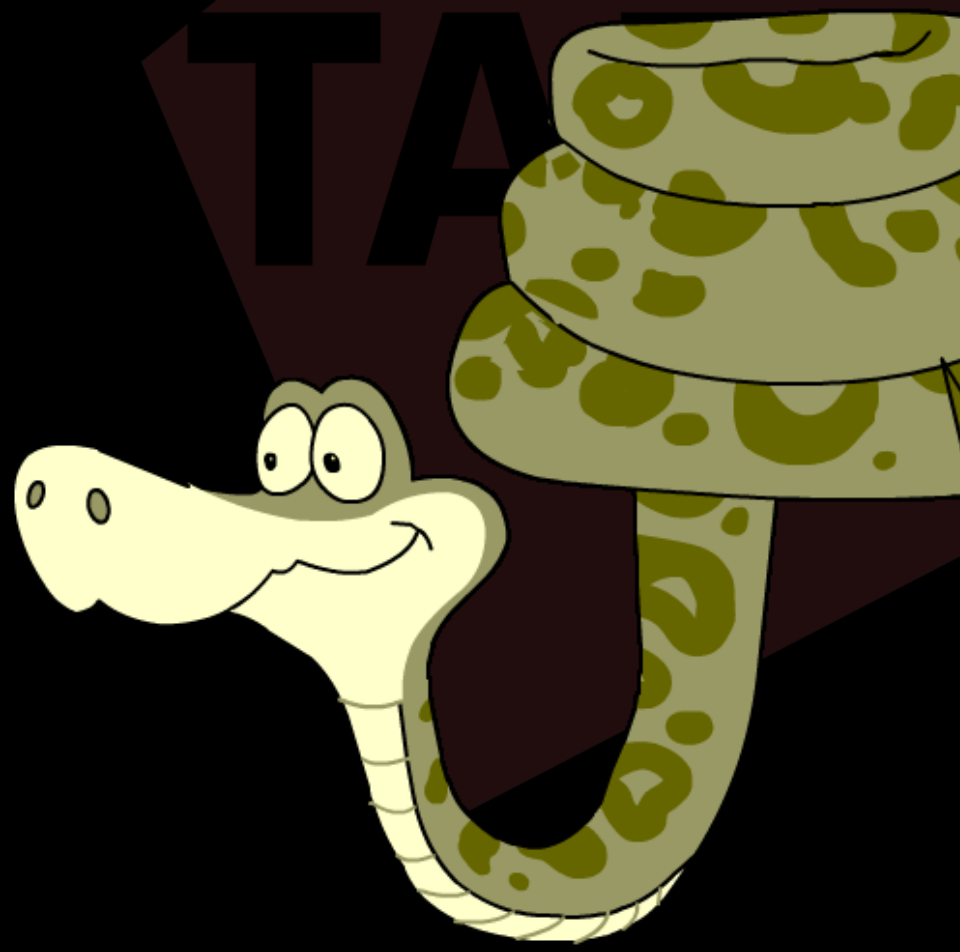
- Open
- Read
- Close
- Write
- Append



Opening and Reading a Text File

To **read the data** stored in a **text file**, you must **open** it first.

Think of it a bit like a book.
You can't read what is inside if you don't open it first.



Opening a Text File

This example opens **temperature.txt** in read mode.

Syntax

```
file = open("temperatures.txt", "r")
```

Opening a Text File

Syntax

```
file  = open("temperatures.txt", "r")
```

Creates an **identifier** (also called a handle) for the file that is to be accessed.

Calls a function that returns a file object.

The 1st argument is the name of the text file to be opened.

The 2nd argument is the mode (read) that the file will be opened.

Opening a Text File: Modes

There are **four modes** for opening a file:

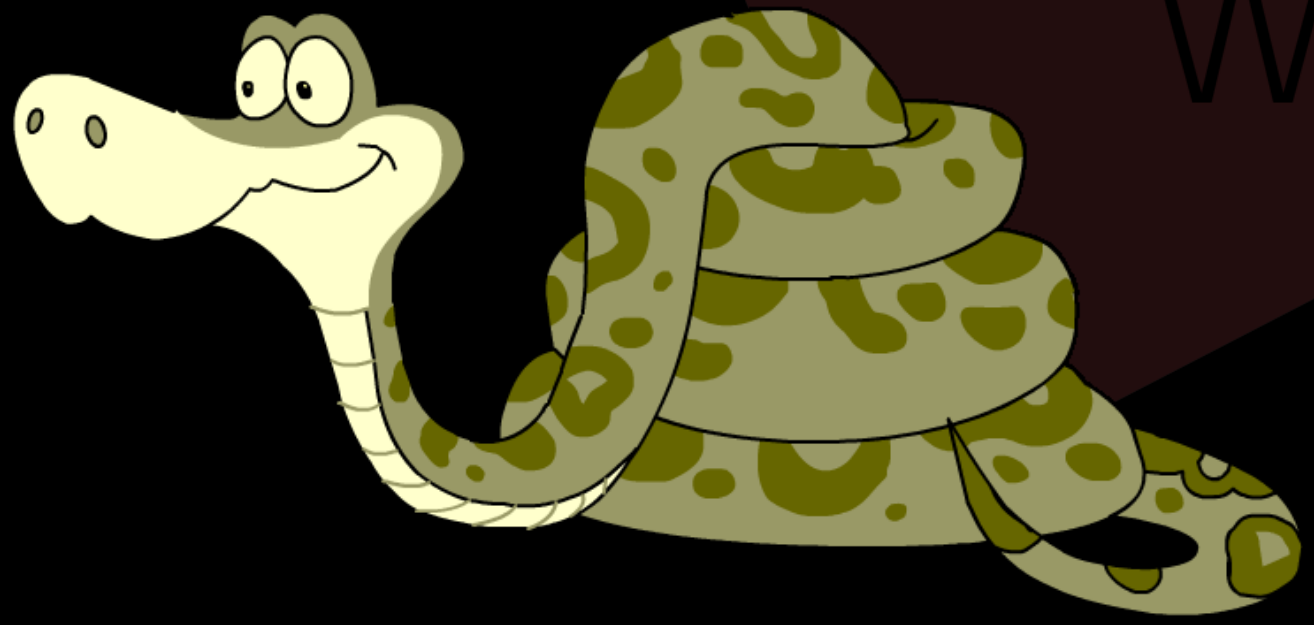
Mode	Description
r	Only reading files
w	Always creates a file writing to the files
a	For creating a file if the file does not exist, adding to an existing file
r+	For reading and writing files

NOTE: Opening a file in write mode will erase any existing file with the same name.

Reading a Text File

Methods to read files include:

- read()
- readline()
- readlines()



Reading a Text File

Method	Description
read()	Reads the entire text file
readline()	Reads a single line from the text file
readlines()	Reads all lines and places them into a list
strip()	Removes the \n from the end of the string
close()	Used when you are finished accessing the text file.

Reading a Text File

This example reads **temperature.txt** and automatically closes the file.

Syntax

```
file =  open("temperatures.txt", "r")

temperatures = file.read()

print(temperatures)
```

Reading a Text File, Line by Line

This example reads the file **temperature.txt** line by line and removes extra whitespace with strip().

Syntax

```
file = open("temperatures.txt", "r")
for line in file:
        print(line.strip())
```

Closing a File

Once you have performed all the tasks that you need to with a file, it is important to **close** it.

This will free up system resources and protect the data.

Syntax

```
file = open("temperatures.txt", "r")
temperatures = file.read()
file.close()
print(temperatures)
```

Let's code!

Read a Text File

- The text file **must be saved** in the **same** location as your **Python file** for the program to work.
- You **must not** use the same variable name as the **file identifier** in your program.
- If file names are **longer than one word**, use **snake case** (underscores between words) i.e: quick_brown_fox.txt.

Read a Text File

Syntax

```
file = open("temperatures.txt", "r")
print(file.read())
```

Output

```
Perth: 37
Melbourne: 20
Hobart: 18
Brisbane: 32
Sydney: 24
```

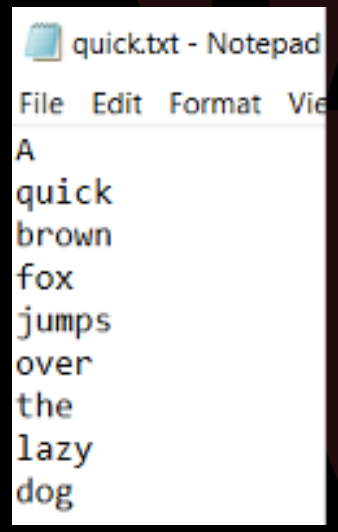
Open and Read a Text File

Step 1: Create a file in notepad, containing the text shown in the **file** example.

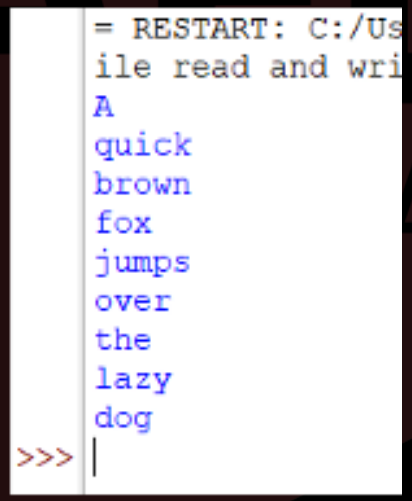
Step 2: Save the file as: quick_brown_fox.txt and save in the same location as your python files.

Step 3: Use and run the code to open, read and display the content of the text file.

File



Output



Reading Lines from a Text File

The readline() method can be used to read a single line from a text file.

Syntax

```
file = open("quick_brown_fox.txt", "r")
print(file.readline())
```

The readline() method will return the line with \n at the end of the string. This adds a new line.

Reading Lines from a Text File

Each time the readline() method is called, the next line in the text file will be returned.

Syntax

```
file = open("quick_brown_fox.txt", "r")
print(file.readline())
print(file.readline())
```

Output

```
A
quick
>>>
```

Reading Lines from a Text File

Another tool for reading lines is the readlines() method.
You can use this to place each line of the text file as an item in a list.

The \n becomes more apparent when you see the data in this way.

Syntax

```
file = open("quick_brown_fox.txt", "r")
quick_brown_fox = file.readlines()
print(quick_brown_fox)
```

Output

```
['A\n', 'quick\n', 'brown\n',
'fox\n', 'jumps\n', 'over\n',
'the\n', 'lazy\n', 'dog']
>>>
```

Note that \n is not added to the final line in the text file.

Reading Lines from a Text File

If you want to populate a list with lines from a text file but you don't want **\n** included in the list, you can write a **for loop** to populate the list instead.

Syntax

```
file = open("quick_brown_fox.txt", "r")
quick_brown_fox = []

for line in file:
    quick_brown_fox.append(line.strip())

print(quick_brown_fox)
```

Output

```
['A', 'quick', 'brown',
'fox', 'jumps', 'over',
'the', 'lazy', 'dog']
>>>
```

Reading Lines from a Text File

You can also **iterate** over a **text file** using a **for loop**.

Syntax

```
file = open("quick_brown_fox.txt", "r")
for line in file:
    print(line)
```

Question: What do you think the output will be when this program is executed?

Reading Lines from a Text File

For every line in the text file, it will print the line, along with an empty line space.

```
A

quick

brown

fox

jumps

over

the

lazy

dog

>>>
```

Reading Lines from a Text File

To remove the extra line space, the strip() method is used.

Syntax

```
file = open("quick_brown_fox.txt", "r")
for line in file:
    print(line.strip())
```

Output

```
A
quick
brown
fox
jumps
over
the
lazy
dog
>>>
```

Default Data Type

When the data is copied from a text file, it will automatically be held using the **string** data type.

Syntax

```
file = open("numbers.txt", "r")
number_list = []

for line in file:
    number_list.append(line.strip())

print(number_list)
```

Output

```
['1', '2', '3', '4', '5', '6', '7',
'8', '9', '10']
>>>
```

Note the ' ' around each number in the list.

Default Data Type

When the data is copied from a text file, it will automatically be held using the **string** data type.

Syntax

```
file = open("numbers.txt", "r")
number_list = []

for line in file:
    number_list.append(int(line.strip()))

print(number_list)
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>>
```

Note that there are no longer quotes around each number.

Writing to a New Text File

When you open a file in **write** mode, it creates a **new file** with the specified file name. If that file **already exists**, then the file will be **overwritten**.

Syntax

```
file = open("numbers.txt", "w")
```

If that file **already exists**, then the file will be overwritten.

Appending a Text File

When you **append** to a file, you are **adding new data** to the **end** of the text file.

The mode character for appending to a file is an **a**.

Syntax

```
file = open("numbers.txt", "a")
```

Appending a Text File

There is no append method for text files.

The **write()** method is used to write and append files.

Syntax

```
file = open("numbers.txt", "a")
file.write("alphabet.txt")
```

What is missing from the program for it to work correctly?

Appending a Text File

Syntax

```
file = open("numbers.txt", "a")
file.write("alphabet")
file.close()
```

Answer

The file **must be closed** for the program to work as expected.

Otherwise, the file will remain open and won't be updated.

Challenge: Appending a Text File

Create a file named **players**.txt

Type the following names and **save**.

Dragon
Nero
Zenith
Rover
Xero

Challenge: Appending a Text File

Append the file to include the name:
Orian

Orian must appear on a new line.

PEP 8 Standards: Files

- Use 4 spaces per indentation level
- Limit all lines to a maximum of 79 characters.

  This helps keep your code readable and manageable.

Files: Best practices

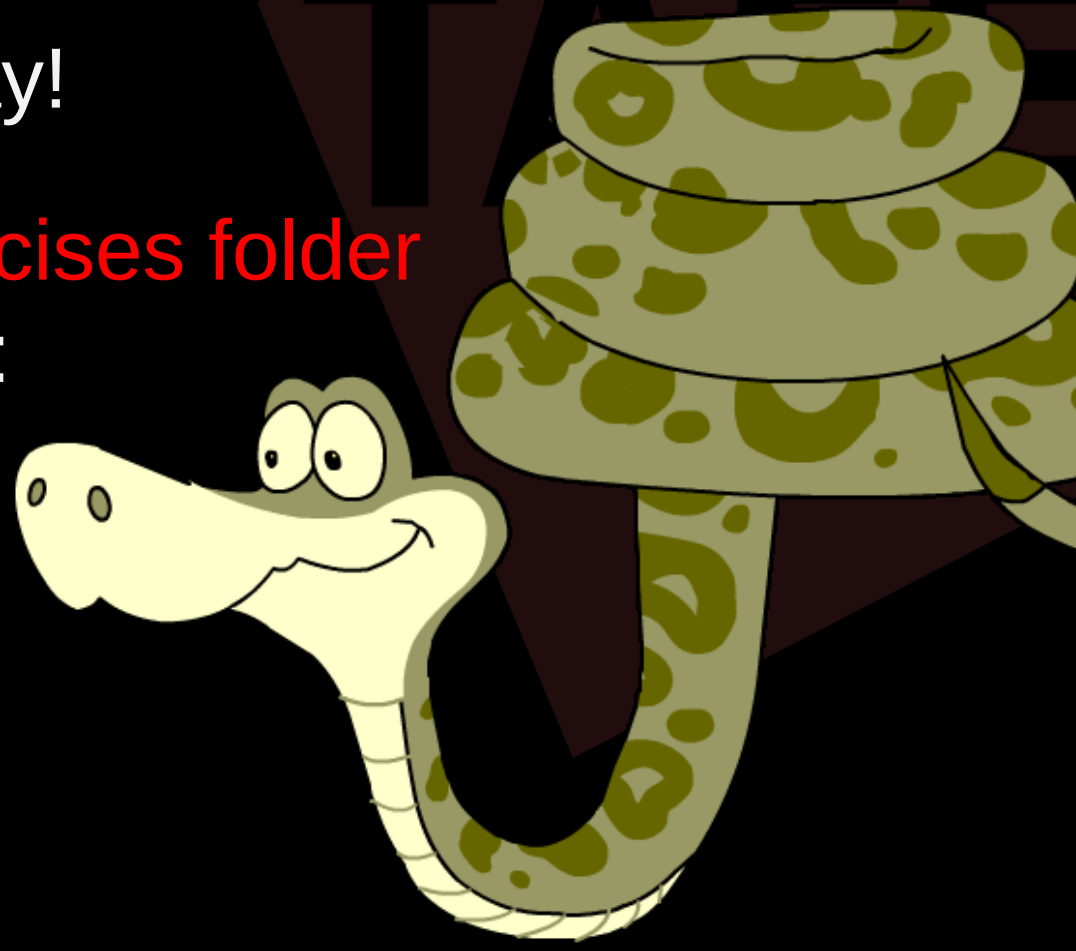
- Always specify the correct mode when opening a file (i.e, **'r'** for reading, **'w'** for writing)
- Use absolute paths to avoid issues with file locations
- Use comments to explain the purpose of file operations and any important details.

Summary

- Opening a file (file handle)
- Reading a file
- Methods: open(), read(), write(), close()
- More PEP8 standards.

Activity to complete

Practise what you have learnt today!

Locate your Week 9 Revision exercises folder and complete the following activity:

1. Text Treasure Hunt